

Backup and Recovery Procedures

Overview

This document outlines the backup and recovery procedures for the Chicken Processing System's Firestore database. These procedures ensure data protection, business continuity, and compliance with data retention requirements.

Backup Strategy

1. Automated Daily Backups

1.1 Full Database Export

```
# Cloud Scheduler job to trigger daily export
gcloud scheduler jobs create pubsub firestore-daily-backup \
  --schedule="0 2 * * *" \
  --topic=firestore-backup \
  --message-body='{"collectionIds": ["users", "products", "orders", "batches",
  "inventory", "compliance", "analytics"]}'
```

1.2 Export Configuration

- **Frequency:** Daily at 02:00 AM (local time)
- **Retention:** 30 days of daily backups
- **Storage:** Cloud Storage bucket `gs://chicken-processing-backups`
- **Format:** Firestore export format (JSON)
- **Compression:** GZIP compression enabled

1.3 Backup Locations

```
gs://chicken-processing-backups/
├─ daily/
│   └─ 2025-01-19/
│       ├── all_namespaces/
│       │   └─ all_kinds/
│       │       ├── all_namespaces_all_kinds.export_metadata
│       │       └─ output-*
│       └─ backup-metadata.json
│   ...
├─ weekly/
│   └─ 2025-W03/
│       ...
└─ monthly/
```

```
└─ 2025-01/  
└─ ...
```

2. Incremental Backups

2.1 Transaction Logs

- Firestore automatically maintains transaction logs
- 7-day retention for point-in-time recovery
- Used for accidental deletion recovery

2.2 Change Streams

```
// Monitor changes for near-real-time backup  
const changeStream = db.collection('orders').watch({  
  includeDocument: true  
});  
  
changeStream.on('change', async (change) => {  
  await backupChange(change);  
});
```

3. Manual Backups

3.1 Pre-Deployment Backups

```
# Manual backup before deployments  
gcloud firestore export gs://chicken-processing-backups/manual/pre-  
deployment-$(date +%Y%m%d-%H%M%S) \  
  --collection-ids=users,products,orders,batches,inventory,compliance,analytics
```

3.2 Schema Change Backups

- Backup before schema migrations
- Backup before data transformations
- Backup before bulk operations

Recovery Procedures

1. Full Database Recovery

1.1 Recovery from Daily Backup

```
# Stop application traffic
gcloud app versions stop v1

# Import from backup
gcloud firestore import gs://chicken-processing-backups/daily/2025-01-19/ \
  --async

# Verify import completion
gcloud firestore operations list

# Restart application
gcloud app versions start v1
```

1.2 Recovery Steps

1. **Assessment:** Determine recovery point and scope
2. **Notification:** Alert stakeholders about recovery process
3. **Isolation:** Isolate affected systems
4. **Restoration:** Execute recovery procedure
5. **Validation:** Verify data integrity
6. **Resumption:** Restore normal operations

2. Partial Recovery

2.1 Collection-Level Recovery

```
# Recover specific collections
gcloud firestore import gs://chicken-processing-backups/daily/2025-01-19/ \
  --collection-ids=users,products
```

2.2 Document-Level Recovery

```
// Manual document recovery from backup
async function recoverDocument(backupPath: string, collection: string,
documentId: string) {
  const backupData = await readBackupDocument(backupPath, collection,
documentId);
  await db.collection(collection).doc(documentId).set(backupData);
}
```

3. Point-in-Time Recovery

3.1 Using Transaction Logs

```
# Recover to specific timestamp
gcloud firestore import gs://chicken-processing-backups/daily/2025-01-19/ \
  --namespace-id='(default)' \
  --async
```

3.2 Recovery Window

- **Last 7 days:** Using Firestore transaction logs
- **Last 30 days:** Using daily backups
- **Beyond 30 days:** Using weekly/monthly archives

Disaster Recovery

1. Regional Failure Recovery

1.1 Multi-Region Setup

- Primary region: **europe-west1** (Belgium)
- Secondary region: **europe-west3** (Frankfurt)
- Cross-region replication for critical data

1.2 Failover Procedure

```
# Update Firestore location
gcloud firestore databases update --location=europe-west3

# Update application configuration
gcloud app deploy --version=v2 --no-promote

# Switch traffic to new region
gcloud app services set-traffic default \
  --splits v2=1 --migrate
```

2. Data Corruption Recovery

2.1 Corruption Detection

```
// Data integrity checks
async function checkDataIntegrity() {
  const checks = [
    checkReferentialIntegrity(),
    checkBusinessRules(),
    checkDataCompleteness(),
    checkConsistency()
  ];
```

```
const results = await Promise.all(checks);
return results.flat();
}
```

2.2 Recovery from Corruption

1. **Identify**: Determine corruption scope
2. **Isolate**: Prevent corruption spread
3. **Restore**: Use last known good backup
4. **Replay**: Apply transaction logs (if safe)
5. **Validate**: Verify recovery success

Testing Procedures

1. Backup Validation

1.1 Automated Validation

```
# Validate backup integrity
gcloud firestore operations describe $(OPERATION_ID) \
  --format="value(done, error)"
```

1.2 Manual Validation

- Monthly restoration tests
- Data completeness verification
- Business rule validation
- Performance testing

2. Recovery Testing

2.1 Quarterly Recovery Drills

```
# Test recovery procedure
./scripts/test-recovery.sh \
  --backup-date=2025-01-01 \
  --collections=users,products \
  --validate-only
```

2.2 Success Criteria

- Recovery time within SLA (4 hours)
- Data loss within RPO (24 hours)

- Application functionality restored
- Performance meets requirements

Monitoring and Alerting

1. Backup Monitoring

1.1 Key Metrics

```
metrics:

- backup_success_rate
- backup_duration_seconds
- backup_size_bytes
- last_successful_backup_timestamp

```

1.2 Alert Conditions

- Backup failure for 2 consecutive days
- Backup duration > 2 hours
- Backup size deviation > 20%
- Last successful backup > 24 hours ago

2. Recovery Readiness

2.1 Health Checks

```
// Daily recovery readiness check
async function checkRecoveryReadiness() {
  return {
    backups: await checkBackupHealth(),
    storage: await checkStorageHealth(),
    network: await checkNetworkHealth(),
    permissions: await checkPermissionHealth()
  };
}
```

2.2 Alert Conditions

- Backup storage quota > 90%
- Backup validation failures
- Permission issues detected
- Network connectivity problems

Compliance and Retention

1. Data Retention Policy

1.1 Retention Periods

- **Daily backups:** 30 days
- **Weekly backups:** 12 weeks
- **Monthly backups:** 36 months
- **Yearly archives:** 7 years
- **Transaction logs:** 7 days

1.2 Legal Requirements

- Financial records: 7 years
- User data: Until account deletion request
- Compliance data: 5 years
- Audit logs: 3 years

2. Data Deletion

2.1 Automated Cleanup

```
# Delete backups older than retention period
gsutil rm -r gs://chicken-processing-backups/daily/2024-12-*/
```

2.2 Secure Deletion

- Cryptographic erasure for sensitive data
- Multiple overwrite passes for compliance data
- Verification of deletion completion

Documentation and Training

1. Runbooks

1.1 Emergency Recovery Runbook

```
# Emergency Recovery Runbook

## Scenario: Complete Database Failure
1. Declare disaster recovery
2. Notify incident commander
3. Execute full recovery procedure
4. Validate recovery success
5. Resume operations

## Scenario: Partial Data Corruption
```

1. Isolate affected data
2. Determine corruption scope
3. Execute partial recovery
4. Validate data integrity
5. Resume normal operations

1.2 Maintenance Runbooks

- Backup verification procedures
- Recovery testing procedures
- Storage cleanup procedures
- Permission management procedures

2. Training

2.1 Team Training

- Quarterly recovery drills
- New team member onboarding
- Procedure updates training
- Tool proficiency assessment

2.2 Documentation Updates

- Procedure changes documented within 24 hours
- Runbooks updated after each incident
- Training materials reviewed quarterly
- Compliance documentation maintained

Cost Management

1. Storage Optimization

1.1 Compression

- Enable GZIP compression for backups
- Implement deduplication where possible
- Archive older backups to cold storage

1.2 Lifecycle Policies

```
{
  "lifecycle": {
    "rule": [
      {
        "action": {"type": "Delete"},
        "condition": {
```

```
    "age": 30,
    "matchesStorageClass": ["STANDARD"]
  },
  {
    "action": {"type": "SetStorageClass"},
    "condition": {
      "age": 90,
      "matchesStorageClass": ["STANDARD"]
    },
    "setStorageClass": "COLDLINE"
  }
]
}
```

2. Cost Monitoring

2.1 Budget Alerts

- Monthly backup storage budget: \$500
- Alert at 80% of budget
- Alert at 100% of budget

2.2 Optimization Reviews

- Quarterly cost review
- Storage efficiency analysis
- Alternative solution evaluation

Conclusion

This backup and recovery strategy provides comprehensive protection for the Chicken Processing System's data while ensuring compliance, cost-effectiveness, and operational readiness. Regular testing, monitoring, and updates will ensure the procedures remain effective as the system evolves.